

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук
должность, уч. степень, звание

подпись, дата

А. В. Бржезовский
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

Хранимые процедуры и функции

по курсу: Методы и средства проектирования
информационных систем и технологий

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Получение навыков создания и использования хранимых процедур и функций в PostgreSQL: вставка и удаление данных с обновлением связанных справочников, агрегатные и табличные функции, использование временных таблиц и управляющих конструкций в процедурном коде.

2 Задание и исходные данные

Вариант 5.

Создайте базу данных для хранения следующих сведений: ВУЗ, студент, группа, факультет, конференция, тема доклада, программа конференции.

Ранее была создана база данных представленная на рисунке 1.

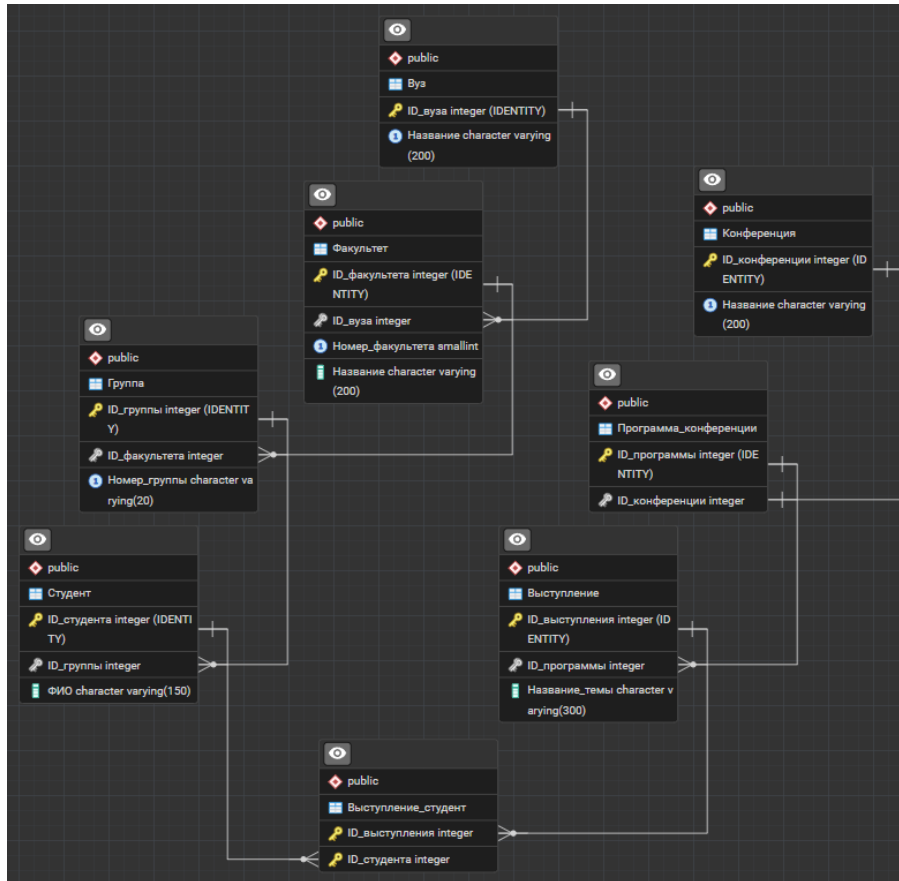


Рисунок 1 – Схема базы данных

На базе данных варианта 5 реализовать хранимые процедуры и функции по п. 6.4: вставка студента с при необходимости созданием строки в справочнике «Группа»; удаление студента с удалением пустой группы; каскадное удаление группы со студентами и участиями в выступлениях; скалярную агрегатную функцию; заполнение временной таблицы статистикой по факультетам. Дополнительно — демонстрация цикла по данным и табличной функции (пп. 6.1 и 6.3).

3 Ход выполнения задания

После подключения к базе данных варианта 5 в PostgreSQL выполнялся скрипт с определениями объектов и примерами вызовов.

Создана процедура добавления студента по ФИО, номеру группы и номеру факультета вуза «ГУАП»: при отсутствии группы с данной парой (факультет, номер группы) в таблицу «Группа» добавляется новая строка, затем вставляется студент; при попытке повторно вставить того же студента в ту же группу выводится предупреждение и вставка не выполняется.

Создана процедура удаления студента по ФИО: удаляются строки участия в «Выступление_студент», затем студент; если в группе не осталось студентов, удаляется строка «Группа».

Создана процедура каскадного удаления группы по номеру группы, номеру факультета и вузу: сначала удаляются участия студентов этой группы, затем студенты, затем группа.

Реализована скалярная функция среднего числа участия в выступлениях на одного студента по выбранному вузу; в среднее входят все студенты вуза, в том числе без выступлений (ноль участия).

Реализована процедура заполнения временной таблицы tmp_faculty_stats: по каждому факультету вуза «ГУАП» подсчитываются число групп, число студентов, число участия в выступлениях и средняя длина названия темы выступлений студентов факультета.

Для иллюстрации управляющих конструкций добавлена процедура с циклом по конференциям и заполнением временной таблицы tmp_conf_loop числом участия в выступлениях по каждой конференции для вуза.

Реализована табличная функция, возвращающая список студентов (ФИО и номер группы), имевших участие в выступлениях программы указанной конференции.

Добавлен отдельный сценарий с использованием CASE: по каждому студенту формируется категория активности в зависимости от числа участия

в выступлениях (нет участия, низкая, средняя, высокая), результат сохраняется во временной таблице.

```
Query Query History

1 call pr_add_student_with_group('Тест ЛР6 Успех', '9901', 1::smallint, 'ГУАП');
2
3 select s."ФИО", g."Номер_группы", f."Номер_факультета", vz."Название"
4 from "Студент" s
5 join "Группа" g on g."ID_группы" = s."ID_группы"
6 join "Факультет" f on f."ID_факультета" = g."ID_факультета"
7 join "Вуз" vz on vz."ID_вуза" = f."ID_вуза"
8 where s."ФИО" = 'Тест ЛР6 Успех';

Data Output Messages Notifications

+ [Icons] SQL

ФИО character varying (150)
Номер_группы character varying (20)
Номер_факультета smallint
Название character varying (200)

1 Тест ЛР6 Успех 9901 1 ГУАП
```

Рисунок 1 – Результат вызова процедуры вставки студента с пополнением справочника «Группа»

```
Query  Query History

1  call pr_delete_student_cleanup_group('Тест ЛР6 Студент');
2
3  -- Проверка: студент удалён
4  select *
5  from "Студент"
6  where "ФИО" = 'Тест ЛР6 Студент';

Data Output  Messages  Notifications

≡+  [File Icon]  [Dropdown]  [Clipboard Icon]  [Dropdown]  [Trash Icon]  [Database Icon]  [Download Icon]  [Line Graph Icon]  SQL

ID_группы  ID_факультета  Номер_группы
[PK] integer  integer  character varying (20)
```

Рисунок 2 – Результат вызова процедуры удаления студента с очисткой
пустой группы

Query Query History

```

1 call pr_add_student_with_group('Тест ЛР6 Гр Студент 1', '9910', 1::smallint, 'ГУАП');
2 call pr_add_student_with_group('Тест ЛР6 Гр Студент 2', '9910', 1::smallint, 'ГУАП');
3
4 -- Удаляем группу каскадно
5 call pr_delete_group_with_students('9910', 1::smallint, 'ГУАП');
6
7 -- Проверка: группа удалена
8 select *
9 from "Группа"
10 where "Номер_группы" = '9910';
11
12 -- Проверка: студенты удалены
13 select *
14 from "Студент"
15 where "ФИО" in ('Тест ЛР6 Гр Студент 1', 'Тест ЛР6 Гр Студент 2');

```

Data Output Messages Notifications

ID студента	ID группы	ФИО
[PK] integer	integer	character varying (150)

Рисунок 3 – Результат вызова процедуры каскадного удаления группы

Query Query History

```

1
2 create or replace function fn_avg_participations_per_student(
3   p_vuz text default 'ГУАП'
4 )
5 returns numeric(12,2)
6 language plpgsql
7 stable
8 as $$
9 declare
10  v_avg numeric(12,2);
11 begin
12  select round(avg(coalesce(t.cnt, 0::bigint))::numeric, 2)
13  into v_avg
14  from (
15    select count(vs."ID_выступления")::bigint as cnt
16    from "Студент" s
17    join "Группа" g on g."ID_группы" = s."ID_группы"
18    join "Факультет" f on f."ID_факультета" = g."ID_факультета"
19    join "Вуз" vz on vz."ID_вуза" = f."ID_вуза"
20    left join "Выступление_студент" vs on vs."ID_студента" = s."ID_студента"
21    where btrim(vz."Название") = btrim(p_vuz)
22    group by s."ID_студента"
23  ) t;
24
25  return coalesce(v_avg, 0::numeric(12,2));
26 end;
27 $$;
28
29
30 select fn_avg_participations_per_student('ГУАП') as "Среднее_число_участий_на_студента";
31

```

Data Output Messages Notifications

Среднее_число_участий_на_студента
1.07

Showing rows: 1 to 1 Page No: 1 of 1

Рисунок 4 – Результат вызова скалярной функции среднего числа участий

Query Query History

```

1 call pr_fill_faculty_stats();
2
3 select *
4 from tmp_faculty_stats
5 order by nomer_fakulteta;

```

Data Output Messages Notifications

nomer_fakulteta	nazvanie_fakulteta	grupp	studentov	uchastiy_v_vystupleniyah	srednyaya_dlina_temy
smallint	character varying (200)	integer	integer	bigint	numeric (10,2)
1	Институт информационных систем и техноло...	4	9	13	33.00
2	Институт радиотехники и телекоммуникаций	1	1	2	34.00
3	Институт электронных приборов и систем	1	2	1	38.00
4	Гуманитарный институт	2	3	0	0.00

Showing rows: 1 to 4

Рисунок 5 – Содержимое временной таблицы tmp_faculty_stats

Query

Query History

1

call pr_demo_conference_participation_loop('ГУАП');

2

3

select *

4

from tmp_conf_loop

5

order by konferenciya;

6

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗑

📥

⬇

📈

SQL

konferenciya

character varying (200)

🔒

uchastiy

bigint

🔒

1

Информатика

7

2

Робототехника и ИИ

7

3

Энергетика и экология

2

Рисунок 6 – Содержимое временной таблицы tmp_conf_loop после цикла

Query

Query History

```
1
2 create or replace function fn_students_by_conference(
3   p_konferenciya text
4 )
5 returns table (
6   fio varchar(150),
7   nomer_grupy varchar(20)
8 )
9 language sql
10 stable
11 as $$
12   select distinct
13     s."ФИО",
14     g."Номер_группы"
15   from "Студент" s
16   join "Группа" g on g."ID_группы" = s."ID_группы"
17   join "Выступление_студент" vs on vs."ID_студента" = s."ID_студента"
18   join "Выступление" v on v."ID_выступления" = vs."ID_выступления"
19   join "Программа_конференции" pc on pc."ID_программы" = v."ID_программы"
20   join "Конференция" k on k."ID_конференции" = pc."ID_конференции"
21   where k."Название" = btrim(p_konferenciya)
22   order by s."ФИО";
23 $$;
24
25 select *
26 from fn_students_by_conference('Информатика')
27 limit 20;
```

Data Output

Messages

Notifications

</

Рисунок 7 – Фрагмент результата табличной функции по конференции

«Информатика»

Query History

```
11  fio varchar(100) not null,
12  номер_группы varchar(30) not null,
13  количество_участий bigint not null,
14  уровень_активности varchar(30) not null
15  ) on commit preserve rows;
16
17  insert into tmp_student_activity_case (
18    fio,
19    номер_группы,
20    количество_участий,
21    уровень_активности
22  )
23  select
24    a."ФИО",
25    g."Номер_группы",
26    count(vs."ID_эсуплнения")::bigint as количество_участий,
27    case
28      when count(vs."ID_эсуплнения") = 0 then 'Нет участий'
29      when count(vs."ID_эсуплнения") between 1 and 2 then 'Низкая'
30      when count(vs."ID_эсуплнения") between 3 and 4 then 'Средняя'
31      else 'Высокая'
32    end as уровень_активности
33  from "Студент" a
34  join "Группы" g on g."ID_группы" = a."ID_группы"
35  join "эсуплнения" f on f."ID_эсуплнения" = g."ID_эсуплнения"
36  join "Вс" vz on vz."ID_вс" = f."ID_вс"
37  left join "Выступление_студент" vs on vs."ID_студента" = a."ID_студента"
38  where btrim(vz."Название") = btrim(g."наз")
39  group by a."ID_студента", a."ФИО", g."Номер_группы"
40  order by a."ФИО";
41  end;
42  $$;
43
44  call pr_fill_student_activity_case('FYAD');
45
46  select *
47  from tmp_student_activity_case
48  order by fio;
```

Data Output

Messages

Notifications

<

Рисунок 8 – Содержимое временной таблицы tmp_student_activity_case (пример применения CASE)

4 Вывод

В ходе работы на базе варианта 5 созданы хранимые процедуры для вставки студента с пополнением справочника «Группа», для удаления студента с очисткой пустой группы и для каскадного удаления группы, реализованы скалярная агрегатная и табличная функции, процедуры с временными таблицами и циклом по конференциям, а также сценарий классификации активности студентов через CASE; получен практический опыт использования объектов PL/pgSQL в PostgreSQL.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
-- 1) вставка с пополнением справочников
create or replace procedure pr_add_student_with_group(
  p_fio text,
  p_nomer_grupy text,
  p_nomer_fakulteta smallint,
  p_vuz text default 'ГУАП'
)
language plpgsql
as $$
declare
  v_id_fakulteta integer;
  v_id_grupy integer;
begin
  if btrim(p_fio) = '' then
    raise exception 'ФИО не может быть пустым';
  end if;

  -- факультет вуза
  select f."ID_факультета"
  into v_id_fakulteta
  from "Факультет" f
  join "Вуз" vz on vz."ID_вуза" = f."ID_вуза"
  where btrim(vz."Название") = btrim(p_vuz)
    and f."Номер_факультета" = p_nomer_fakulteta;

  if v_id_fakulteta is null then
```

```
raise exception 'Факультет с номером % для вуза «%» не найден',  
p_nomer_fakulteta, p_vuz;  
end if;
```

```
-- группа; при отсутствии создаём
```

```
select g."ID_группы"  
into v_id_grupy  
from "Группа" g  
where g."ID_факультета" = v_id_fakulteta  
and g."Номер_группы" = btrim(p_nomer_grupy);
```

```
if v_id_grupy is null then  
insert into "Группа" ("ID_факультета", "Номер_группы")  
values (v_id_fakulteta, btrim(p_nomer_grupy))  
returning "ID_группы" into v_id_grupy;  
end if;
```

```
-- не дублировать студента с тем же ФИО в той же группе
```

```
if exists (  
select *  
from "Студент" s  
where s."ID_группы" = v_id_grupy  
and s."ФИО" = btrim(p_fio)  
) then  
raise notice 'Студент «%» уже есть в группе %', btrim(p_fio),  
btrim(p_nomer_grupy);  
return;  
end if;
```

```
insert into "Студент" ("ID_группы", "ФИО")
```

```

values (v_id_grupy, btrim(p_fio));
end;
$$;

-- call pr_add_student_with_group('Пробный Студент
ЛР6','9999',1::smallint,'ГУАП');

-- 2) удаление с очисткой справочника «Группа»

create or replace procedure pr_delete_student_cleanup_group(
  p_fio text
)
language plpgsql
as $$
declare
  v_id_student integer;
  v_id_grupy integer;
begin
  select s."ID_студента", s."ID_группы"
  into v_id_student, v_id_grupy
  from "Студент" s
  where s."ФИО" = btrim(p_fio);

  if v_id_student is null then
    raise notice 'Студент «%» не найден', p_fio;
    return;
  end if;

  delete from "Выступление_студент" vs
  where vs."ID_студента" = v_id_student;

```

```

delete from "Студент" s
where s."ID_студента" = v_id_student;

if not exists (
    select *
    from "Студент" s2
    where s2."ID_группы" = v_id_grupy
) then
    delete from "Группа" g
    where g."ID_группы" = v_id_grupy;
end if;
end;
$$;

-- call pr_delete_student_cleanup_group('Пробный Студент ЛР6');

-- 3) каскадное удаление группы

create or replace procedure pr_delete_group_with_students(
    p_nomer_grupy text,
    p_nomer_fakulteta smallint,
    p_vuz text default 'ГУАП'
)
language plpgsql
as $$
declare
    v_id_grupy integer;

```

```

begin
  select g."ID_группы"
  into v_id_grupy
  from "Группа" g
  join "Факультет" f on f."ID_факультета" = g."ID_факультета"
  join "Вуз" vz on vz."ID_вуза" = f."ID_вуза"
  where btrim(vz."Название") = btrim(p_vuz)
    and f."Номер_факультета" = p_nomer_fakulteta
    and g."Номер_группы" = btrim(p_nomer_grupy);

  if v_id_grupy is null then
    raise notice 'Группа не найдена';
    return;
  end if;

  delete from "Выступление_студент" vs
  using "Студент" s
  where vs."ID_студента" = s."ID_студента"
    and s."ID_группы" = v_id_grupy;

  delete from "Студент" s
  where s."ID_группы" = v_id_grupy;

  delete from "Группа" g
  where g."ID_группы" = v_id_grupy;
end;
$$;

-- call pr_delete_group_with_students('9999', 1::smallint, 'ГУАП');

```

-- 4) скалярная функция — среднее число участия на студента (все студенты вуза, в т.ч. с нулём участия)

```
create or replace function fn_avg_participations_per_student(  
  p_vuz text default 'ГУАП'  
)  
returns numeric(12,2)  
language plpgsql  
stable  
as $$  
declare  
  v_avg numeric(12,2);  
begin  
  select round(avg(coalesce(t.cnt, 0::bigint))::numeric, 2)  
  into v_avg  
  from (  
    select count(vs."ID_выступления")::bigint as cnt  
    from "Студент" s  
    join "Группа" g on g."ID_группы" = s."ID_группы"  
    join "Факультет" f on f."ID_факультета" = g."ID_факультета"  
    join "Вуз" vz on vz."ID_вуза" = f."ID_вуза"  
    left join "Выступление_студент" vs on vs."ID_студента" = s."ID_студента"  
    where btrim(vz."Название") = btrim(p_vuz)  
    group by s."ID_студента"  
  ) t;  
  
  return coalesce(v_avg, 0::numeric(12,2));  
end;
```

\$\$;

```
select fn_avg_participations_per_student('ГУАП') as  
"Среднее_число_участий_на_студента";
```

-- 5) статистика во временной таблице по факультетам

```
create or replace procedure pr_fill_faculty_stats()  
language plpgsql  
as $$
```

```
begin  
drop table if exists tmp_faculty_stats;
```

```
create temp table tmp_faculty_stats (  
    nomer_fakulteta smallint not null,  
    nazvanie_fakulteta varchar(200),  
    grupp int not null,  
    studentov int not null,  
    uchastiy_v_vystupleniyah bigint not null,  
    srednyaya_dlina_temy numeric(10, 2)  
) on commit preserve rows;
```

```
insert into tmp_faculty_stats (  
    nomer_fakulteta,  
    nazvanie_fakulteta,  
    grupp,  
    studentov,  
    uchastiy_v_vystupleniyah,  
    srednyaya_dlina_temy  
)
```



```

select
  f."Номер_факультета",
  f."Название",

  (
    select count(*)::int
    from "Группа" g
    where g."ID_факультета" = f."ID_факультета"
  ),

  (
    select count(*)::int
    from "Студент" s
    join "Группа" g on g."ID_группы" = s."ID_группы"
    where g."ID_факультета" = f."ID_факультета"
  ),

  coalesce(
    (
      select count(*)
      from "Выступление_студент" vs
      join "Студент" s on s."ID_студента" = vs."ID_студента"
      join "Группа" g on g."ID_группы" = s."ID_группы"
      where g."ID_факультета" = f."ID_факультета"
    ),
    0
  ),

  coalesce(
    (

```

```

select round(avg(char_length(v."Название_темы"))::numeric, 2)
from "Выступление_студент" vs
join "Студент" s on s."ID_студента" = vs."ID_студента"
join "Группа" g on g."ID_группы" = s."ID_группы"
join "Выступление" v on v."ID_выступления" = vs."ID_выступления"
where g."ID_факультета" = f."ID_факультета"
),
0::numeric
)

from "Факультет" f
join "Вуз" vz on vz."ID_вуза" = f."ID_вуза"
where btrim(vz."Название") = 'ГУАП'
order by f."Номер_факультета";
end;
$$;

call pr_fill_faculty_stats();

select *
from tmp_faculty_stats
order by nomer_fakulteta;

-- 6) управляющие конструкции - цикл по конференциям, заполнение отчёта

create or replace procedure pr_demo_conference_participation_loop(
    p_vuz text default 'ГУАП'
)
language plpgsql

```

```

as $$
declare
  r record;
  v_cnt bigint;
begin
  drop table if exists tmp_conf_loop;

  create temp table tmp_conf_loop (
    konferenciya varchar(200) not null,
    uchastiy bigint not null
  ) on commit preserve rows;

  for r in
    select k."Название" as nazvanie
    from "Конференция" k
    order by k."Название"
  loop
    select count(*)::bigint
    into v_cnt
    from "Выступление_студент" vs
    join "Студент" s on s."ID_студента" = vs."ID_студента"
    join "Группа" g on g."ID_группы" = s."ID_группы"
    join "Факультет" f on f."ID_факультета" = g."ID_факультета"
    join "Вуз" vz on vz."ID_вуза" = f."ID_вуза"
    join "Выступление" v on v."ID_выступления" = vs."ID_выступления"
    join "Программа_конференции" pc on pc."ID_программы" =
v."ID_программы"
    join "Конференция" k2 on k2."ID_конференции" = pc."ID_конференции"
    where btrim(vz."Название") = btrim(p_vuz)
    and k2."Название" = r.nazvanie;

```

```

insert into tmp_conf_loop (konferenciya, uchastiy)
values (r.nazvanie, coalesce(v_cnt, 0));
end loop;
end;
$$;

```

```

call pr_demo_conference_participation_loop('ГУАП');

```

```

select *
from tmp_conf_loop
order by konferenciya;

```

-- 7) табличная функция — студенты конференции (ФИО и номер группы)

```

create or replace function fn_students_by_conference(
  p_konferenciya text
)
returns table (
  fio varchar(150),
  nomer_grupy varchar(20)
)
language sql
stable
as $$
select distinct
  s."ФИО",
  g."Номер_группы"
from "Студент" s

```

```

join "Группа" g on g."ID_группы" = s."ID_группы"
join "Выступление_студент" vs on vs."ID_студента" = s."ID_студента"
join "Выступление" v on v."ID_выступления" = vs."ID_выступления"
join "Программа_конференции" pc on pc."ID_программы" =
v."ID_программы"
join "Конференция" k on k."ID_конференции" = pc."ID_конференции"
where k."Название" = btrim(p_konferenciya)
order by s."ФИО";
$$;

```

```

select *
from fn_students_by_conference('Информатика')
limit 20;

```

-- 8) *ситуация с CASE — категории активности студентов по числу
участий*

```

create or replace procedure pr_fill_student_activity_case(
    p_vuz text default 'ГУАП'
)
language plpgsql
as $$
begin
    drop table if exists tmp_student_activity_case;

    create temp table tmp_student_activity_case (
        fio varchar(150) not null,
        nomer_grupy varchar(20) not null,
        kolichestvo_uchastiy bigint not null,
        uroven_aktivnosti varchar(30) not null

```

```
) on commit preserve rows;
```

```
insert into tmp_student_activity_case (
```

```
    fio,
```

```
    nomer_grupy,
```

```
    kolichestvo_uchastiy,
```

```
    uroven_aktivnosti
```

```
)
```

```
select
```

```
    s."ФИО",
```

```
    g."Номер_группы",
```

```
    count(vs."ID_выступления")::bigint as kolichestvo_uchastiy,
```

```
    case
```

```
        when count(vs."ID_выступления") = 0 then 'Нет участия'
```

```
        when count(vs."ID_выступления") between 1 and 2 then 'Низкая'
```

```
        when count(vs."ID_выступления") between 3 and 4 then 'Средняя'
```

```
        else 'Высокая'
```

```
    end as uroven_aktivnosti
```

```
from "Студент" s
```

```
join "Группа" g on g."ID_группы" = s."ID_группы"
```

```
join "Факультет" f on f."ID_факультета" = g."ID_факультета"
```

```
join "Вуз" vz on vz."ID_вуза" = f."ID_вуза"
```

```
left join "Выступление_студент" vs on vs."ID_студента" = s."ID_студента"
```

```
where btrim(vz."Название") = btrim(p_vuz)
```

```
group by s."ID_студента", s."ФИО", g."Номер_группы"
```

```
order by s."ФИО";
```

```
end;
```

```
$$;
```

```
call pr_fill_student_activity_case('ГУАП');
```

```
select *  
from tmp_student_activity_case  
order by fio;
```